SecondChance: Measuring Recovery after Evaluator Correction: State-bound endpoints, strong neutral controls and commit-resolved utility

Research Forge · 18 October 2026 · Working paper v13

Conference-style development preprint, 2026-10-18. Authored deterministic execution evidence; no natural LLM recovery or native-method superiority demonstrated. v13 is an editorial/source-review revision of preserved v11/v12 experiments.

Abstract

Correcting an evaluator changes a score; it does not by itself restore the decisions, integrated artifact or written lesson that the faulty evaluator helped produce. We develop an executable research instrument that separates immutable-output rescoring, a fresh candidate invocation and a fresh audit of the integrated current state. Twelve authored signed-rational-addition history templates yield 168 fixed-pool policy trajectories, with a strong neutral early-stop comparator and a separate fixed-artifact lesson contrast. Rescoring improves saved scores in all twelve templates while restoring none of their current states. At budget five, neutral joint repair matches the proposed lineage-first procedure's four terminal recoveries while using fewer authored work units and causing less collateral loss. We then freeze every trajectory and freshly audit its distinct committed states: 292 local subprocesses produce 876 panel outputs. Sixteen trajectories have a positive initial-recovery area of 0.2 that the earlier five-point left-step measure recorded as zero. Conversely, coarse observation overstates unaffected-task utility in 32 trajectories. A commit-geometry bound covers every observed discrepancy. These are constructed deterministic execution results, not LLM performance or natural correction frequencies. The instrument identifies measurement and control requirements for a future model study while leaving natural-history recovery, long-term evolutionary recurrence and native-method superiority unmeasured.

1 Introduction

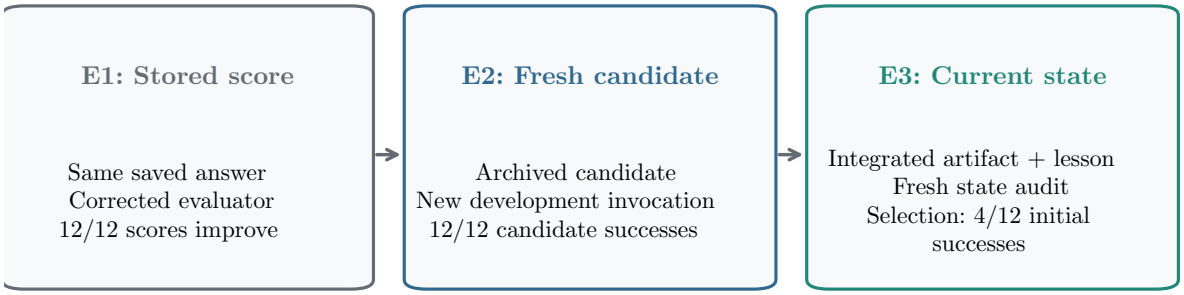
An evaluator patch can reverse yesterday's rejection while leaving today's system broken. Consider a correct signed-arithmetic implementation that was rejected because an evaluator compared its answer against an absolute value. The incumbent remains the wrong program; a later, valid output-format fix may already have been applied; and a lesson written from the rejection may still recommend the wrong procedure. Rescoring the archived answer changes the historical verdict. It does not execute an integration, preserve the later fix or withdraw the lesson. A deployment decision therefore requires a behavioral endpoint on the actual current state, rather than a corrected score alone.

This distinction matters for systems that update persistent textual state. SkillOpt gates bounded skill edits on validation and retains rejected edits as feedback [1]; Reflexion conditions future trials on feedback-derived reflective memory [2]. Self-Refine and CRITIC improve outputs through different feedback loops [3,4], while SCoRe learns correction behavior through online reinforcement learning [6]. These mechanisms motivate inspecting decisions and memory after an evaluator correction. They do not by themselves answer the narrower identification question studied here: after the requirement remains fixed and its evaluator is corrected, what evidence establishes that an archived alternative has actually repaired the integrated current state?

We make that question executable before testing it on language models. SecondChance is a small deterministic research instrument with an immutable requirement, authored faulty/corrected evaluators, a fixed archive, logged state commits and fresh state audits. It separates E1, rescoring the same saved output; E2, freshly invoking a candidate; and E3, auditing the integrated artifact with its surviving lesson and valid later changes. Figure 1 depicts these endpoints. The construction permits exact inspection and falsification of a measurement contract, but supplies no estimate of natural evaluator faults, LLM recovery probability or method superiority.

We ask three linked questions. RQ1: can saved-score and fresh-candidate success disagree with integrated-state recovery? RQ2: does lineage-first joint repair beat a strong neutral early-stop procedure under the same authored caps? RQ3: when recovery is summarized over intervention cost, how much can a coarse observation grid hide? Twelve authored histories yield 168 nested policy-budget paths. The neutral procedure ties terminal recovery and is better on cost and collateral loss at budget five. A follow-up freezes every path and makes 292 new state audits. It reveals both missed recovery and understated collateral degradation, with discrepancies covered by a conditional commit-geometry

bound. Thus the paper's contribution is a falsifiable recovery measurement and control protocol, including negative policy evidence, rather than a demonstrated new LLM recovery algorithm.



Different endpoints: rescoring alone yields 0/12 current-state recoveries.

Figure 1: Three endpoints after the same evaluator correction. E1 changes a saved verdict; E2 invokes an archived candidate; E3 newly executes the integrated state. At budget eight, selection arms have 12/12 development-positive candidates but 4/12 initial-state recoveries, while rescoring alone restores 0/12. Counts describe twelve authored histories, not language-model performance.

Contributions.

- An executable distinction between saved-score correction, fresh candidate success and integrated-state recovery, with state/response hashes and retained denominators.
- Matched-history policy procedures, a strong neutral early-stop control, and a same-artifact lesson fork that prevents a selector difference from being mislabeled as a memory-only effect.
- A fresh committed-state follow-up of all 168 frozen trajectories, separating observation resolution from policy improvement and audit cost from controller cost.
- A conditional reconstruction argument and a bounded-utility commit/grid error bound, including a counterexample to treating coarse left-step recovery area as universally conservative.

2 Related Work

We organize comparisons by the decision each method changes, the feedback it may access and the object it evaluates. This avoids treating every reflection or verification method as an interchangeable repair baseline. Our nineteen references support particular design choices; their reported benchmark numbers are not pooled with our executor results.

2.1 Correction, selection and persistent memory

Self-Refine uses one model to generate, critique and refine an output [3]; CRITIC incorporates tool interaction when validating and revising an output [4]. Huang et al. examine intrinsic reasoning self-correction without external feedback [5]. Their scope is narrower than all correction, especially in light of later learned correction strategies such as SCoRe [6]. Our evaluator correction is an explicit external event, and our current study runs no language model. These papers motivate controlling feedback and update rights; neither their successes nor their limitations can be imported as our empirical result.

Verifier-based selection ranks generated candidate solutions [7]. SkillOpt instead optimizes a persistent skill document through bounded edits and a validation gate, with rejected-edit feedback retained for later updates [1]. Consequently, retaining rejected candidates is not our novelty claim. We audit what a changed evaluator requires after those decisions: immutable old outputs, a common current-state fork, actual integration, preserved later changes and a fresh integrated-state outcome. The fixture is a fixed-pool experiment; generating a new corrective skill remains a separate comparison.

Reflexion provides the closest motivation for examining feedback-derived lessons [2]. Zep maintains temporally scoped facts and invalidates inconsistent edges when information changes [8]. Our setting holds the requirement fixed and corrects its erroneous implementation; world-state change and evaluator correction have different approval conditions. We therefore test a same-artifact retain/suspend lesson fork and leave native Reflexion, Zep and SkillOpt reproduction unmeasured.

Family	Object changed	Feedback contract	Comparison here
Self-Refine / CRITIC [3,4]	Current response	Self feedback / tool-derived critique	Design context; not run
Intrinsic correction / SCoRe [5,6]	Reasoning response / learned correction policy	No external feedback / online training	Different training and feedback rights
Verifiers / SkillOpt [7,1]	Candidate choice / persistent skill	Verifier ranking / validation-gated edits	Fixed pool and correction eligibility specified
Reflexion / Zep [2,8]	Reflective text / temporal fact graph	Task feedback / changing information	Same-artifact lesson fork; native results NR
Neutral early-stop	Integrated fixture state	Same corrected development grader	Executed strong control

Table 1: Mechanism comparison, not an external performance leaderboard. NR means not reported because the method was not executed.

2.2 Evaluation validity and executable correctness

Proxy optimization can improve a development score without improving a separately evaluated objective [9]. Model-selection bias and adaptive holdout reuse make evaluation access part of the method, rather than an implementation detail [10,11]. These motivate freezing candidate pools, audit inputs and disclosure boundaries. Our researcher can inspect every fixture label; controller non-use is documented but is not independent adjudication or a secret final test.

EvalPlus shows why finite code tests can miss incorrect implementations [17]. SWE-bench supplies repository issue-resolution tasks that could support a richer future environment [18], but neither resource automatically labels an old evaluator as wrong under an unchanged requirement. Such labels, the historical rejected edits and their downstream lessons must be collected separately. ReAct's interleaving of reasoning and environment actions also makes clear why a prompt or program hash alone may omit relevant agent state [19].

2.3 Resource curves and reproducible comparisons

Henderson et al. emphasize nondeterminism and reporting variability [12]; Agarwal et al. address uncertainty in aggregate few-run comparisons [13]. We retain history-level denominators and avoid confidence intervals that would falsely turn designed templates into sampled natural histories. Local process counts and wall durations are execution resources, not additional independent trials.

Performance profiles compare distributions of a declared performance metric [14], and COCO makes problem instances, targets and function-call runtime explicit [15,16]. Those ideas motivate reporting both terminal behavior and an intervention-cost curve. Our integrated utility can decrease after a repair, unlike a retained best-so-far objective; hitting a target once does not guarantee its subsequent persistence. The area and geometry bound below are our conditional measurement argument, not a theorem imported from numerical-optimization benchmarks.

3 Problem Formulation

The unit of comparison is a frozen history under a verified correction to a fixed requirement. The state includes the executable artifact and any active lesson that affects routing. Correction eligibility, selection success and current recovery are separate predicates.

3.1 Correction eligibility and behavioral endpoints

The unchanged requirement G is exact signed addition of a/b and c/d. The response grammar permits a bounded signed integer, decimal or nonzero-denominator fraction after FINAL=. Numerically equivalent rational forms are accepted, so FINAL=-2/3 and FINAL=-4/6 denote the same value. Parsing uses a bounded grammar and Python Fraction; response text is never evaluated as code. Non-numeric values, zero denominators, excessive numeric lengths and arbitrary prefixes are rejected. Approximate decimal values are not silently accepted as exact rationals.

The authored defect replaces the exact signed target with its absolute value. Under the negative development sum, an absolute-value program is accepted while an exact signed program is rejected. The accepted correction removes that absolute-value operation while preserving the task and valid-answer contract. Old outputs and old decisions are appended to, never replaced by corrected scores. A change in requested sign, task identity, accepted alternatives or evaluation objective would instead define a new task or a disputed correction and must not be counted as the same correction event.

Correction eligibility in this fixture follows from the authored arithmetic contract, not independent human review. In an empirical scientific-label setting, the fixed requirement, counterexample, old assertion, corrected assertion and valid alternatives would need a separate adjudication procedure. A model critic agreeing with the correction would not provide that independence. Ambiguous annota-

tions, requirements changes and unsupported corrections should remain pending rather than become useful-rejection evidence.

Endpoint E1 is V_new applied to the exact same saved response bytes; it measures historical score change. Endpoint E2 is an actual fresh candidate invocation on permitted development data; it measures what that candidate newly does under its own state. Endpoint E3 is a fresh audit of the integrated current-state checkpoint, including retained lessons and routing. E2 can be positive for a development-memorizing candidate while E3 fails on a different input. E1 has no state-transition semantics at all.

$$G(x) = \frac{a}{b} + \frac{c}{d} \tag{1}$$

$$V_{\text{old}}(y, x) = I(\text{parse}(y) = |G(x)|) \tag{2}$$

$$V_{\text{new}}(y, x) = I(\text{parse}(y) = G(x)) \tag{3}$$

$$\Delta_{\text{saved}} = V_{\text{new}}(y_{\text{saved}}, x) - V_{\text{old}}(y_{\text{saved}}, x) \tag{4}$$

3.2 Complete state and decision lineage

A checkpoint contains a program operation, the lesson text and its active/absent/suspended status, and a valid later change that must survive repair. The latter fixes the FINAL= output prefix and the JSON trace format. Candidate artifacts encode exact rational addition, absolute-value addition, development-input memorization, integer truncation or a constant zero response. Each candidate runs in a separate local Python process. Its request bytes, response bytes, state hash, worker hash, return code and observed process duration are recorded.

The history records the candidate archive, saved outputs, old accept/reject events, incumbent and any actually written fixture lesson. The fixed authored lesson writer can record the instruction to fall back to the absolute-value solver for opposite-sign addition. This is not a model reflection. The initial audit uses negative same-sign terms; the later panel uses opposite-sign terms with a negative total. The constructed lesson can therefore preserve initial success after integration yet break success on the later input. All those task values and the writer rule are visible to the researcher.

Lineage is also explicitly authored. An aligned hint points to the useful exact candidate; a misleading hint points to the development-memorizing candidate. Every policy receives the same hint and archive. The neutral policy chooses not to use the hint for its ordering. This design exposes the consequences of reliable versus unreliable localization; it does not learn a localization model or estimate the prevalence of good historical lineage.

A saved rescore and a candidate invocation do not themselves mutate the incumbent. Only the logged integration and lesson-suspension events can change this fixture state. Integration verifies the candidate artifact and preserved valid-change hash. A common parent hash binds all policy forks. The pre-lesson checkpoint and final checkpoint are retained separately. This complete state definition is a testable assumption, not a property guaranteed for general agents with filesystem state, latent memory, tools or external services.

Freeze the requirement, faulty/corrected assertions, archive bytes, old decisions, incumbent and written lesson.
Fork the same initial state for each already declared policy and budget.
Charge each rescore, fresh candidate execution, integration and lesson action; retain unused allowance.
Commit state only after integration or lesson treatment and store the state/parent/artifact hashes.
Audit isolated state copies on the fixed panels; keep audit responses outside policy selection.
Retain no-useful-candidate, no-written-lesson, no-gain, invalid, timeout and missing-audit outcomes.

Algorithm 2: Research instrument, not a model-driven autonomous repair algorithm.

4 Method: A Recovery Audit Contract

The instrument runs matched intervention procedures and observes state-bound outcomes. It does not use a critic's agreement as evidence of correctness. A policy result is identified only relative to a recorded state, allowed information, intervention budget, audit panel and failure/missingness rule.

4.1 Endpoint denominators and the lesson contrast

A current initial recovery requires success on the fresh initial-input audit of the integrated state. The main all-started burden counts no initial recovery or failure on the later-input panel. It keeps templates without useful rejected candidates in the denominator. Conditional persistence failure divides the number of initial successes that fail later by the number of initial successes; zero initial successes yield null, not zero. Joint initial-success/later-failure frequency and total later failure are separate all-template quantities.

The label later refers to a different input executed against the same final checkpoint. No further agent evolution, model training, new selection or calendar-time drift occurs between the initial and later panels. Thus the stored strict_recurrence field denotes cross-panel maintenance failure in the fixture. It is not a measured long-term evolutionary recurrence rate. A future recurrence study must specify the subsequent evolution process and preserve its additional decisions, exposure and resources.

Unaffected-task performance is paired before and after intervention. Absolute failure after repair and incremental loss from the same baseline are different quantities. This matters because selecting a development-memorizing program can destroy an unaffected positive-addition capability even when the corrected development score is perfect. Templates with such collateral losses remain in every relevant table.

The fixed-artifact lesson contrast is built only after selection and integration have happened once. The exact pre-lesson bytes are copied into retain and suspend branches, preserving program artifact, valid later changes, routing and allowance. Selection is not rerun. This controls the integration mediator that would otherwise differ between whole procedures. In a general model experiment, treatment assignment, context length, allowed correction knowledge and memory retrieval would also need explicit control. Here the mechanism is visibly authored in the worker, so its contrast verifies implementation rather than discovering an LLM memory mechanism.

$$B_{\text{all}} = \left(\frac{1}{N}\right) \sum_{h=1}^{N} I(R_h = 0 \text{ or } P_h = 0) \tag{5}$$

$$F_{\text{cond}} = \frac{\sum_h I(R_h = 1 \text{ and } P_h = 0)}{\sum_h I(R_h = 1)} \tag{6}$$

$$\Gamma = Y_{11} - Y_{10} - Y_{01} + Y_{00} \tag{7}$$

4.2 Cost-indexed recovery utility

Version 11 observes the current state at policy-cost coordinates 0, B/4, B/2, 3B/4 and B and computes a normalized left-step area. A terminal success can therefore have zero recorded area. For example, an initially failing state integrated successfully at cost four under cap five is still failing at the last left endpoint 3.75 and successful only at the final observation five. The resulting zero is correct for the declared coarse statistic, but it is not the area of the complete committed-state path.

Let c_i be every committed-state cost, including zero and a terminal boundary B. Let s_i be the state in force after c_i, held until the next commit. If the log contains every state mutation, the worker is a pure deterministic function of that state and the fixed audit input, and each positive-duration state has actually been freshly audited, then the state-path area is identified by the observed step sum. This is a conditional statement about this executor and panel. It is not a claim that one output identifies a stochastic agent's expected capability.

The argument is direct: partition the cost axis at every complete state commit. The executor and input do not change within each resulting interval, so its binary output is constant there. A fresh observed value for that state supplies the interval's utility, and integration over the finite partition gives A_C. Audits are not inferred from saved-score positives or candidate identifiers. A missing positive-duration audit leaves its area contribution unknown in [0,width/B]. A missing state that exists only at the endpoint B can leave the area defined while the terminal score remains missing.

The cost axis excludes the common external audit envelope. Adding audit spending retroactively to the controller's five- or eight-unit axis would change the estimand and possibly the feasible repair procedure. We instead report actual audit subprocess counts and elapsed time as a separate resource vector. These are local process measurements, not model tokens, billed dollars or a paired hardware-speed benchmark.

$$A_G = \left(\frac{1}{B}\right) \sum_{j=0}^{J-1} (g_{j+1} - g_j) U(s(g_j)) \tag{8}$$

$$A_C = \left(\frac{1}{B}\right) \sum_{i=0}^{K-1} (c_{i+1} - c_i) U(s_i) \tag{9}$$

$$\Delta_A = A_C - A_G \tag{10}$$

4.3 Commit reconstruction and an error bound

For each coarse cell [g_j,g_(j+1)), let t_j be its first strictly interior state-changing commit. If there is no such commit, set t_j to the right boundary. A commit at a coarse grid point is already observed

there, and a metadata event leaving the state bytes unchanged creates no new exposure interval. Until t_j the state equals the coarse left-endpoint state. Afterwards each bounded utility can differ from that left value by at most one. Summing those potentially exposed lengths proves the bound below without a monotonicity assumption.

The bound is sharp in a one-change cell: a utility jump from zero to one, or from one to zero, at the first interior commit persists through the cell. For the budget-five integration at four, the exposed interval is [4,5), yielding E=1/5. A coarse initial-area value of zero and a freshly observed commit-aware value of 1/5 attain the bound. Grid-aligned integration at four under budget eight has no initial-area discrepancy because four is already an observed grid point.

Monotone nondecreasing utility implies that a left-step coarse area cannot exceed the complete state-path area. General repair does not guarantee monotonicity. Collateral degradation supplies the reverse-sign counterexample: a state with unaffected utility one can switch to zero between observations, making the coarse area too large. Nor does observing a successful final state rule out an unobserved transient failure or recovery. Full commit logs and fresh values are necessary for the stronger reconstruction claim.

Unknown state-changing events, an incomplete routing/environment snapshot, a fresh same-state response that differs from the earlier response, a missing positive-duration state audit, or a discrepancy exceeding the geometric bound falsifies the corresponding identification step. Such cases should suspend the point estimate; a critic's agreement cannot repair missing observations. Our self-checks include missing positive-width audits, missing terminal-only observations, a decreasing-utility counterexample and unchanged-state commits.

$$E = \left(\frac{1}{B}\right) \sum_{j=0}^{J-1} (g_{j+1} - t_j) \tag{11}$$

$$|\Delta_A| \leq E \tag{12}$$

$$A_{\text{lower}} = A_{\text{observed}}, \quad A_{\text{upper}} = A_{\text{observed}} + \frac{W_{\text{unknown}}}{B} \tag{13}$$

5 Experimental Setup

We distinguish the original policy experiment from a later measurement experiment on its frozen paths. Every number below comes from the two retained local execution result files, not an external paper or a new model call. The editorial v13 cycle changes the exposition, figure specification and reviewed references; it does not create new policy trials.

5.1 Authored cohort, policies and budgets

The cohort comprises twelve authored templates: eight contain a previously rejected exact candidate and four contain no useful rejected candidate. Candidate position, hint alignment and presence of the written fixture lesson vary. Each archive has four candidates. The same six unique arithmetic inputs serve development, two additional-input checks, the initial audit, later-input audit and unaffected-input audit. Reuse of those inputs across policies does not create independent tasks or histories.

The old receipt labels one call kind independent_authored_qualification. That historical identifier is retained for byte compatibility, but its meaning is restricted here to two additional authored numeric inputs that the controller does not read. The same author, source code and arithmetic rule determine every input and answer. Those checks are neither independently authored correction qualification nor a secret held-out evaluation. Programmatic non-use by the controller is weaker than an access-controlled external audit boundary.

All seven policies receive work-unit caps of five and eight. Rescoring costs one authored unit, a fresh candidate execution two, integration one and lesson treatment one. Selection reserves enough allowance for its fresh execution/integration and, where assigned, lesson action. It stops at the first development-positive candidate, freshly executes that candidate and integrates on development success. Neutral and lineage-first policies share this acceptance rule. Equal caps do not force equal spending. No candidate is newly generated during these fixed-pool procedures.

The selection-by-lesson factorial uses no repair, selection only, lesson only and selective joint repair with V_new in every cell. This is a whole-procedure contrast: data-query ordering, failed selection, reservations and integration can differ across cells. Neutral early-stop and neutral joint are essential additional controls; exhaustive replay alone would be a weak comparator. The rescore-only arm is an endpoint control rather than a candidate recovery procedure.

Policy	Archive/selection action	Lesson action
No repair	None	Retain
Rescore only	Score immutable stored responses	Retain
Neutral early-stop	Fixed archive order, first development-positive candidate	Retain
Selection only	Hint-first order, same acceptance rule	Retain
Lesson only	Keep incumbent	Suspend if active
Neutral joint	Neutral early-stop with lesson reserve	Suspend if active
Selective joint	Hint-first selection with lesson reserve	Suspend if active

Table 2: Same starting history and allowed information; the realized costs and outputs remain outcomes.

5.2 Frozen-trajectory committed-state re-audit

Before the follow-up, the runner verifies every artifact hash in the prior v11 execution receipt and pins the old worker implementation. It fixes all 168 trajectory files, twelve history records, the three audit inputs and its own analysis code. Every event is replayed from the recorded initial state. Integration checks the archive artifact and preserved valid change; lesson treatment checks its supporting text hash. Reconstructed pre-lesson and final snapshots must exactly equal the authenticated original snapshots. An unrecognized event is rejected because it could hide a state mutation.

For each trajectory separately, every distinct committed-state hash is submitted to a new local worker subprocess on the initial, later and unaffected inputs. There is no audit cache across trajectories. A repeated byte-identical state within a trajectory reuses its new observed result only under the declared deterministic-worker assumption; this reduces redundant subprocesses without pretending that reused observations are independent samples. Every request, stdout, stderr, worker hash, process return code and elapsed duration is preserved.

The new observations are compared with the original coarse observations wherever they bind the same state hash. Drift suspends commit-aware point estimation. The primary analysis compares A_C with the retained A_G on the same frozen policy path, checks the geometric bound for all three panels and reports every initiated template. No policy sees the fresh audit outputs, no candidate is regenerated, no lesson treatment is changed and no failed policy is removed.

The first follow-up attempt stopped after one state subprocess because the adapter initially omitted the reused area function's explicit budget argument. That failed directory, its frozen plan, exact failed analysis-code bytes and failure record are preserved. The corrected complete attempt uses a new directory. Its 292 state subprocesses are reported separately from that one completed subprocess in an incomplete attempt. No incomplete attempt is counted as a successful trajectory or discarded from the execution history.

Verify prior receipt hashes and freeze all trajectory/history inputs before fresh execution.
Reconstruct every state commit and validate initial, pre-lesson and final checkpoint hashes.
Within each frozen trajectory, newly execute every distinct committed state on the same three panels.
Bind fresh outputs to request/state/worker hashes and check agreement with prior same-state observations.
Compute coarse and committed-state areas, missingness bounds and the geometry error bound.
Report all 168 paths, separate audit resources and retained failed attempts; do not alter the policies.

Algorithm 3: A post-freeze measurement study, not a second policy-performance trial.

5.3 Analysis unit and resource accounting

The first experiment contains twelve authored history templates, seven procedures and two caps: $12 \times 7 \times 2 = 168$ trajectories. They reuse six unique authored arithmetic inputs. The state follow-up reuses three of those panel inputs, auditing 292 distinct states within trajectories and producing 876 outputs. Neither repeated policies, panel outputs nor subprocess invocations enlarge the independent history sample. The template construction varies availability of a useful rejection, lesson presence, archive order and aligned/misleading lineage; it does not sample those factors from deployment.

The action schedule charges one unit per rescore, two per candidate invocation, one per integration and one per lesson treatment. These are authored work units. We report realized spending under equal caps, because forcing equal spending would remove early stopping as an outcome. Common external audits have a separate ledger: the first experiment used 840 coarse-checkpoint calls plus 24 same-artifact memory audits; its total 1,048 subprocesses also includes candidate/qualification work. The successful follow-up uses 292 state subprocesses. No provider costs or hardware-calibrated efficiency ratios are inferred.

All selected tables are generated from frozen summaries, with all twelve started histories in each row. A zero conditional-recovery denominator is null. No-repair, rescore-only and lesson-only arms

remain even when their recovery area is zero. The incomplete first follow-up attempt remains outside successful aggregates and is explicitly reported in the reproducibility appendix.

6 Results and Analysis

We report the endpoint separation and policy negative result before the new area analysis. This ordering prevents a more informative measurement from being mistaken for an improved controller.

6.1 Saved-score improvement is not current-state recovery

The completed v11 run contains 1,048 local subprocess calls and no model requests. It reports 168 policy-budget trajectories, not an empirical sample size of 168 independent histories. All twelve templates improve at least one saved score after correction, but the rescore-only procedure leaves all current states unrecovered. Fresh development executions also expose endpoint non-equivalence: twelve successful candidate invocations in each unrestricted selection arm yield only four initial-input successes. The fixture's memorizing candidate explains that constructed generalization failure.

At budget eight, neutral joint and selective joint both have four initial successes and four later-input successes out of twelve. Without lesson suspension the corresponding selection procedures have four initial successes and two later-input successes, giving a conditional maintenance-failure value of 2/4. Procedures with no initial successes have null conditional maintenance failure. These results do not demonstrate a general advantage of lineage-first repair.

B=8 procedure	Saved-score improvement	Fresh dev candidate success	Initial state success	Later-input state success
no repair	0/12	No call	0/12	0/12
rescore only	12/12	No call	0/12	0/12
neutral early stop	12/12	12/12	4/12	2/12
selection only	12/12	12/12	4/12	2/12
lesson only	0/12	No call	0/12	0/12
neutral joint	12/12	12/12	4/12	4/12
selective joint	12/12	12/12	4/12	4/12

Table 3: Version 11 authored executor results. Different columns are different endpoints; none is a language-model accuracy estimate.

6.2 A strong neutral baseline limits the policy claim

The terminal-count tie does not justify declaring all procedures equivalent: realized cost and collateral capability remain outcomes. Table 3 shows the joint procedures. Under cap five, neutral joint has the same four initial and four later-input successes, spends three rather than five authored units on average, and preserves the unaffected task in all twelve histories rather than four. Under cap eight, terminal counts and collateral loss tie, with lower authored spending for selective joint. This reversal is why we do not claim general lineage-first superiority.

The selected procedure follows an authored hint. In the six aligned-hint histories its later-input success is 4/6 versus 2/6 for neutral joint; in the six misleading-hint histories it is 0/6 versus 2/6. Aggregate equality is partly a consequence of the deliberately balanced construction. The same-artifact lesson fork provides a narrower mechanism contrast: two of twelve retain/suspend pairs improve on the later input and ten do not. The programmed lesson rule explains this result; it is not a discovered causal property of real model memory.

Cap	Procedure	Initial / later successes	Mean work units	Unaffected loss
5	neutral joint	4/12 ; 4/12	3.00	0.000
5	selective joint	4/12 ; 4/12	5.00	0.667
8	neutral joint	4/12 ; 4/12	5.67	0.667
8	selective joint	4/12 ; 4/12	5.00	0.667

Table 4: Executed authored paths, twelve histories per row. Work units are not tokens, money or CPU. The budget-five neutral advantage is retained.

6.3 Fresh state audits reveal two directions of sampling error

The successful follow-up reconstructs all 168 trajectories and makes 292 fresh state subprocess calls, producing 876 panel outputs with no failed calls. Every comparable same-state output agrees with the prior observation. The earlier coarse protocol made 840 checkpoint audit calls on these trajectories; its additional 24 fixed-artifact memory-audit calls are excluded from that schedule comparison. The 292 count exploits deterministic within-trajectory state identity, so it is not a universal audit-cost advantage for stochastic agents. Observed subprocess duration sums to approximately 57.30 seconds in this execution, without a controlled latency comparison.

Sixteen trajectories have positive initial-area discrepancies, each exactly 0.2; the other 152 have zero difference. All positive cases are budget-five procedures with successful integration at cost four. Per procedure, four of twelve templates change from coarse initial mean zero to committed-state mean 1/15. At budget eight the corresponding initial mean remains 1/6 because integration is grid-aligned. Each positive initial discrepancy attains its trajectory's geometry bound. Terminal success counts remain exactly those of v11.

Later-input utility exposes a separate timing issue. At budget five, joint repair can suspend a harmful lesson only at the cap five. That increases the final later-input success count from two to four without adding positive-duration utility after that last treatment. The committed-state later-input mean is consequently 1/30 for both single-selection and joint procedures at that budget. At budget eight, joint repair's later-input area changes from 1/8 under coarse observation to 7/48 under the full commit path, because lesson suspension occurs at five between the coarse points four and six.

Unaffected-task utility has negative discrepancies in 32 trajectories and zero in 136. For example, at budget five the selective-joint mean drops from coarse unaffected utility 1 to committed-state utility 13/15, while neutral joint remains at 1. At budget eight, neutral joint changes from 5/6 to 3/4 on that panel. All panel discrepancies satisfy their geometry bounds. These reverse-sign cases refute a universal claim that coarse left-step evaluation is conservative. The policy ties, misleading-lineage failures and budget-five neutral-baseline advantage are preserved rather than reinterpreted as a new repair improvement.

Budget/procedure	Initial A_G	Initial A_C	Later A_G	Later A_C	Fresh audits / prior coarse
5 / neutral early-stop	0	1/15	0	1/30	24 / 60
5 / selection only	0	1/15	0	1/30	24 / 60
5 / neutral joint	0	1/15	0	1/30	22 / 60
5 / selective joint	0	1/15	0	1/30	30 / 60
8 / neutral early-stop	1/6	1/6	1/12	1/12	24 / 60
8 / selection only	1/6	1/6	1/12	1/12	24 / 60
8 / neutral joint	1/6	1/6	1/8	7/48	30 / 60
8 / selective joint	1/6	1/6	1/8	7/48	30 / 60

Table 5: New observations of the frozen authored paths, twelve templates per row. No-repair, rescore-only and lesson-only initial/later areas remain zero at both budgets. Their audit counts are retained in results.json, not dropped.

6.4 Reading the area and terminal outcomes together

Figure 2 isolates the cost-four integration under cap five. The last coarse left endpoint is 3.75, so a successful [4,5) interval contributes nothing to the coarse statistic and one fifth to the complete committed-state area. Across the frozen cohort, this occurs on sixteen initial-utility paths. Figure 3 contrasts initial and unaffected panels for neutral and selective joint repair; the latter can also deteriorate between observations. Terminal counts do not change when the audit schedule is refined.

The new calls agree with the earlier responses whenever they bind identical state bytes. Every observed panel-area discrepancy lies within the geometry bound. These checks support the internal reconstruction claim for this worker and panel; they do not certify that three inputs characterize a general program or a language model. A final successful state can still have zero post-treatment area if treatment completes exactly at the cap, and an apparently stable coarse curve can still hide a collateral failure.

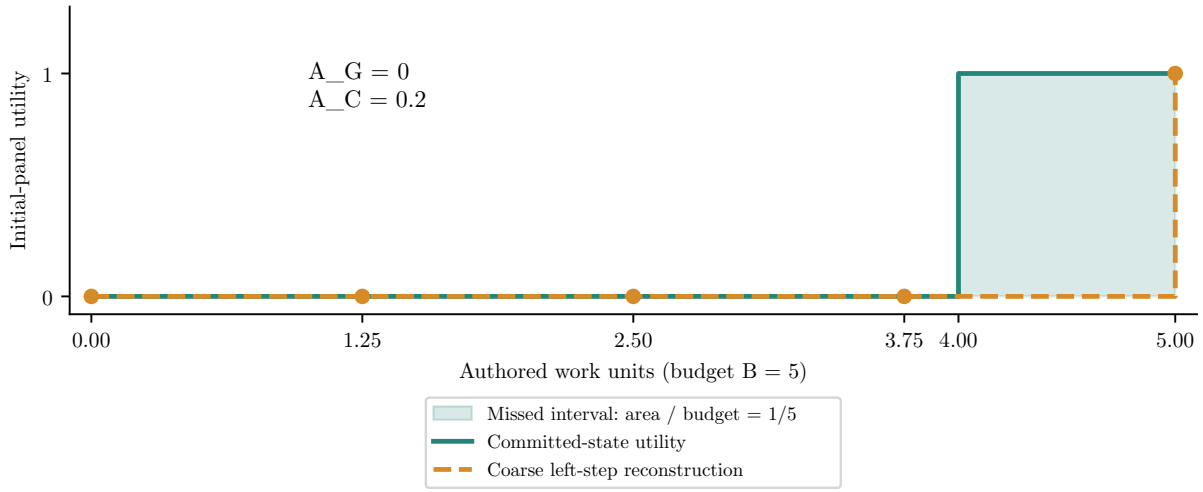


Figure 4: A constructed cost-indexed state path, verified by fresh state execution. Integration at cost four under cap five yields committed-state area 0.2 but coarse left-step area zero. The shaded missed interval attains the geometric bound. Assumptions are complete commits, pure deterministic execution and a fresh observation of every positive-width state.

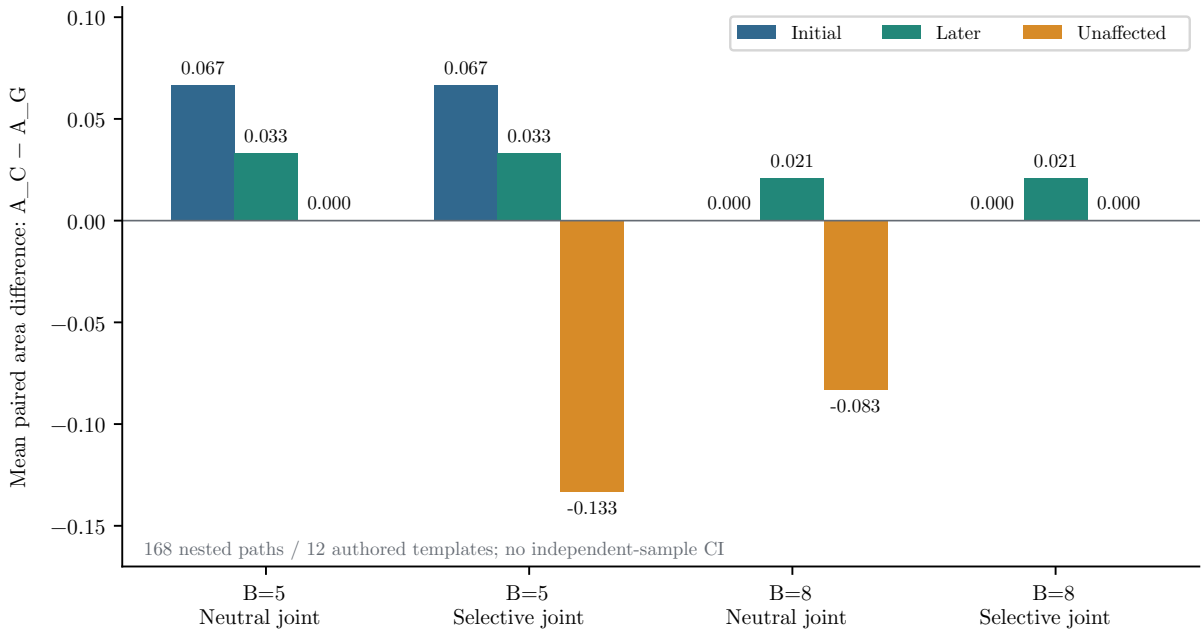


Figure 5: Paired coarse and committed-state mean utility on fixed authored paths, twelve history templates per procedure/cap. Initial utility can be underestimated; unaffected utility can be overestimated. Across all 168 nested paths, sixteen initial areas increase and thirty-two unaffected areas decrease. These are descriptive finite-cohort counts, not independent population samples or LLM effects.

7 Discussion and Limitations

The practical lesson is a decision rule: do not promote a recovered score, candidate or skill until the integrated state has been observed under the corrected contract and preserved capabilities have been checked. This is a measurement requirement supported by constructed counterexamples. It is not evidence that the proposed lineage-first procedure should replace the neutral control.

7.1 Identification assumptions and threats to validity

Construction is the dominant limitation. One author defines arithmetic inputs, candidate programs, the evaluator defect, lineage and harmful lesson routing. Eight histories contain a useful rejected candidate and four do not. Those proportions, the aligned/misleading balance and the recovery counts are experimental choices and consequences, not deployment incidence estimates. The same few inputs recur throughout; researcher-visible audit answers prevent any independent final-test interpretation.

Complete-state reconstruction is conditional. Files, retrievers, caches, tool responses, external services and latent or stochastic model state could invalidate the claim that utility stays fixed between logged commits. A same-state disagreement, an unknown state mutation, a missing positive-duration audit

or a violation of the bound must suspend a point estimate. For stochastic agents, state-level repeated sampling and a declared expectation/uncertainty model are needed; one observed answer does not identify capability.

The later-input panel runs the same final state on a different authored input. No further training, candidate evolution or calendar-time drift occurs. The historical field strict_recurrence is therefore a maintenance-failure statistic across these panels, not long-term evolutionary recurrence. Similarly, exact local hashes support integrity checks, not external authorship signatures, scientific correctness or conference acceptance.

7.2 A staged real-model confirmation study

The executed negative findings narrow the proposal. Lineage-first ordering should be a conditional hypothesis about trustworthy decision support, with neutral fallback and its cost included. A useful-rejected-candidate gate before integration may address development memorization, but its extra queries, exposure and selection opportunity must be charged. Reusing the final audit labels for that gate would change the evaluation rather than demonstrate improvement. Current-only model retuning should be added as a separately budgeted generation procedure, not described as a native comparison already completed here.

A model feasibility study needs an actually generated versioned candidate archive, real old-grader accept/reject decisions and any lesson the model really wrote. Before correction, freeze the common history, active artifact, preserved later changes and model/runtime/settings. The correction needs independent approval under an unchanged contract. Assign protected task roles and stop rules before generating outcomes; keep no-useful-reject, no-written-lesson, disputed-correction, missing-archive, failed-integration and incomplete-audit histories visible in the flow.

Matched-policy forks must have common data rights and envelopes for selector calls, candidate invocations, input/output tokens, grader calls, integration, lesson editing and audits. A complete model state may include filesystem, tool responses, retrieval indices and environment snapshots beyond a prompt hash. For stochastic outputs, repeated state audits estimate a panel expectation with nested sampling uncertainty; one deterministic fixture output supplies neither a population confidence interval nor a stochastic capability curve. New correction families and evolution prefixes must be clustered rather than treating repeated responses as independent histories.

Only after those conditions are met can the hypotheses concern real recovery or continued-evolution recurrence. A native SkillOpt, Reflexion or Zep comparison additionally needs the original algorithm's declared objective, environment, reset and resource contracts. Until then, our findings concern endpoint design, a fully inspectable constructed failure mechanism and measurement resolution. Model critic feedback, named methodological lenses and successful checksum verification do not close those scientific gates.

7.3 Benchmark choice and falsifiable next hypotheses

A feasible first extension is an access-controlled program-edit history over a fixed subset of real issue-resolution tasks. SWE-bench can motivate the environment [18], while EvalPlus motivates additional behavioral tests [17]; neither alone supplies correction labels. Before model use, construct an explicit inclusion flow: unchanged requirement, authenticated old rejection, independent correction approval, available candidate archive, observable current state, and approved audit rights. Preserve failed, disputed and absent-useful-candidate histories in the flow. Repository licenses, runtime feasibility and evaluation exposure must be checked before acquiring or executing a selected dataset.

The primary superiority hypothesis should be conditional on reliable lineage and tested against neutral early-stop, neutral joint repair and a separately budgeted current-only regeneration procedure. A useful-candidate gate must use permitted development data and pay for its extra queries; reusing final audit labels would change the task. The primary recovery endpoint should be fresh integrated-state utility on independently held histories, with preserved-capability loss and total model/tool resources as co-primary costs. The stopping rule, smallest relevant effect and interval procedure should be set before model outcomes, rather than obtained by treating 168 nested paths as independent observations.

Long-term recurrence requires another declared evolution prefix after repair, with new decisions, lessons and resource/exposure records. It is a separate future experiment. The current design will be rejected or revised if neutral procedures retain their advantage on the declared target, if lineage fails to localize useful rejections, if the state snapshot cannot support reproducible audits, or if additional intervention cost outweighs any independently measured behavioral gain.

8 Conclusion

Evaluator correction, candidate success and current-state recovery are different endpoints. Second-Chance makes the distinction executable and exposes a strong neutral control that ties recovery and can avoid unnecessary cost and collateral loss. Its frozen-path follow-up shows that coarse utility sampling can miss both recovery and degradation; under complete deterministic state logs, freshly observed commits identify a stepwise area with an explicit geometry bound. The supported contribution is an auditable measurement and comparison contract with retained negative results. Natural LLM recovery, native-method superiority and long-term recurrence remain unmeasured.

Acknowledgements

This work was created as part of the No-More-Slops group activities at Pseudo Lab.

References

[1] Yifan Yang et al. SkillOpt: Executive Strategy for Self-Evolving Agent Skills. arXiv:2605.23904v2, 2026.
[2] Noah Shinn et al. Reflexion: language agents with verbal reinforcement learning. NeurIPS 2023.
[3] Aman Madaan et al. Self-Refine: Iterative Refinement with Self-Feedback. NeurIPS 2023.
[4] Zhibin Gou et al. CRITIC: Large Language Models Can Self-Correct with Tool-Interactive Critiquing. ICLR 2024.
[5] Jie Huang et al. Large Language Models Cannot Self-Correct Reasoning Yet. ICLR 2024.
[6] Aviral Kumar et al. Training Language Models to Self-Correct via Reinforcement Learning. ICLR 2025.
[7] Karl Cobbe et al. Training Verifiers to Solve Math Word Problems. arXiv:2110.14168v2, 2021.
[8] Preston Rasmussen et al. Zep: A Temporal Knowledge Graph Architecture for Agent Memory. arXiv:2501.13956v1, 2025.
[9] Leo Gao, John Schulman, and Jacob Hilton. Scaling Laws for Reward Model Overoptimization. ICML 2023, PMLR 202:10835–10866.
[10] Gavin C. Cawley and Nicola L. C. Talbot. On Over-fitting in Model Selection and Subsequent Selection Bias in Performance Evaluation. JMLR 11:2079–2107, 2010.
[11] Cynthia Dwork et al. Generalization in Adaptive Data Analysis and Holdout Reuse. arXiv:1506.02629v2, 2015.
[12] Peter Henderson et al. Deep Reinforcement Learning that Matters. AAAI 2018; author version arXiv:1709.06560.
[13] Rishabh Agarwal et al. Deep Reinforcement Learning at the Edge of the Statistical Precipice. NeurIPS 2021; author version arXiv:2108.13264v4.
[14] Elizabeth D. Dolan and Jorge J. Moré. Benchmarking optimization software with performance profiles. Mathematical Programming 91:201–213, 2002.
[15] Nikolaus Hansen et al. COCO: A Platform for Comparing Continuous Optimizers in a Black-Box Setting. arXiv:1603.08785v4, 2020.
[16] Nikolaus Hansen, Anne Auger, Dimo Brockhoff, Dejan Tušar, and Tea Tušar. COCO: Performance Assessment. arXiv:1605.03560v1, 2016.
[17] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. Is Your Code Generated by ChatGPT Really Correct? Rigorous Evaluation of Large Language Models for Code Generation. NeurIPS 2023.
[18] Carlos E. Jimenez et al. SWE-bench: Can Language Models Resolve Real-world Github Issues? ICLR 2024.
[19] Shunyu Yao et al. ReAct: Synergizing Reasoning and Acting in Language Models. ICLR 2023; author version arXiv:2210.03629v3.

A Reproducibility and failed attempts

The v11 source is projects/research_progress_v11_20261018/second_chance/lab.py and its original execution is runs/research_progress_v11_20261018_01/second_chance. The new runner is commit_audit.py in this manuscript project. Execute it with –out and a fresh directory; it verifies prior sealed inputs, invokes only the existing authored local worker and refuses an existing output path. No model endpoint, external network request, installed dependency, active skill or original run is modified.

The complete follow-up is runs/research_manuscripts_v12_20261018_01/second_chance/commit_audit_02. Its plan.json fixes inputs and assumptions; traces/ records reconstructed commits and state-bound fresh observations; invocation_receipts.json binds subprocess requests/outputs; results.json retains every policy/budget denominator; receipt.json hashes every output and consumed input. commit_audit_01 preserves the failed adapter attempt and its exact old analysis bytes. Unsigned hashes supply local integrity rather than publisher identity, provenance signatures or scientific certification.

The read-only verify.py recomputes input/output hashes, state reconstruction, deterministic output equivalence, audit costs, coarse and commit-aware areas, geometric bounds and aggregates without invoking subprocesses. Seven authored metric self-checks cover a tight one-fifth discrepancy, grid-aligned commits, positive-width missingness, terminal-only missingness, decreasing utility and unchanged-state events. Four completed-run checks require all 168 reconstructions, concordance, geometry bounds and all twelve initiated templates. Additional tamper/path rejection validation is recorded separately; no fabricated independent experiment is created by a test assertion.

B Proof details and missing observations

Fix an audit input and suppose every state-changing event is logged; the executor is a pure deterministic function of the complete recorded state; and every state occupying positive cost width has a fresh valid audit. Partition [0,B) at the state commits. State and input remain constant on each half-open interval, so utility is constant and its integral equals utility times interval width. Summing yields A_C. Endpoint-only states require a separate terminal observation even though their contribution to area is zero.

For a coarse interval [g_j,g_(j+1)), a grid-aligned commit is included in the state at its left endpoint. If the first strictly interior state-changing commit is t_j, actual and left-sampled utilities are equal on [g_j,t_j). On the remaining interval their difference has absolute value at most one. The absolute integral error is therefore bounded by its length; the triangle inequality over cells and division by B gives E. Extra commits can cancel error but cannot violate this bound. With utilities in [L,H], multiply E by H-L. A single persistent jump at t_j attains the bound.

For missing positive-width observations, let W_unknown be their total width and sum only known contributions as A_observed. With utilities in [0,1], area lies in [A_observed, A_observed + W_unknown/B]; retain null for an unidentified point estimate. Missing endpoint-only audits do not create area width and do not authorize filling in a terminal score. Unknown commits or incompletely captured environments invalidate the partition itself, rather than merely increasing a conventional standard error.

C Source review and reporting boundaries

SOURCES.json records nineteen primary-source reviews performed on 2026-10-18, their reviewed sections, short retained excerpts, excerpt hashes and claim links. A hash binds the retained excerpt and local review record, not a publisher webpage, DOI identity or an unsigned scientific truth claim. Most methodological comparisons use official abstracts; selected full-text methods were additionally inspected for SkillOpt, Reflexion, Self-Refine, CRITIC and Zep. The Huang arXiv HTML retrieval failed, so its scope is restricted to the official ICLR abstract. No arbitrary remote PDF was downloaded.

The v12 manuscript cited a mutable SkillOpt-Sleep companion preview. It is omitted from this revision because it was not necessary to identify a new scientific comparison beyond the version-pinned SkillOpt method. Reflexion conference metadata and arXiv v4 have different author lists; the bibliography uses the official proceedings edition and identifies the separate method-review edition explicitly in SOURCES.json. No published performance number is used as a locally reproduced result. No venue-specific submission format or acceptance claim is implied by this neutral conference-style preprint.

D Source inspection boundaries

[1] Author arXiv abstract and HTML Sections 3.1/3.5; bounded edits, validation gate and rejected-edit feedback only. No native execution or superiority comparison.

[2] Official conference abstract; selected method Section 3 of arXiv:2303.11366v4 reviewed separately. Different author lists are not merged.

[3] Official abstract and arXiv:2303.17651v2 Section 2; same-model generation/feedback/refinement loop. Published effects are not imported.

[4] Official abstract and arXiv:2305.11738v4 Section 3/Algorithm 1; tool-derived feedback and output revision, not retrospective archive repair.

[5] Official abstract only: intrinsic reasoning self-correction without external feedback. HTML v3 retrieval failed; no full-text or universal impossibility claim.

[6] Official abstract only: SCoRe online multi-turn RL and self-generated correction traces. No assertion of failure under later methods.

[7] Author arXiv abstract: generate candidates and select using a verifier. No GSM8K execution or external performance transfer.

[8] Author abstract and HTML Section 2.2.3: temporal validity/edge invalidation. World changes are distinguished from fixed-requirement correction.

[9] Official abstract: proxy versus gold-reward optimization. No numerical scaling law is claimed for our deterministic fixture.

[10] Official abstract: overfitting a finite-sample selection criterion and biased performance evaluation. Motivates protected audit roles; no theorem transfer.

[11] Author abstract: adaptive reuse can overfit a holdout. No privacy mechanism or guarantee is implemented here.

[12] Author abstract and official AAAI search result; concerns nondeterminism, variability and experimental reporting. No RL experiment here.

[13] Author abstract: finite-run uncertainty and aggregate evaluation. We do not attach population intervals to designed templates.

[14] Publisher abstract/bibliography only: distribution functions of a performance metric. Not claimed as a recovery-area theorem.

[15] Author abstract: problem instances, target values and objective-call runtime. Resource-accounting motivation only; no optimizer benchmark.

[16] Author abstract: any-time evaluation using objective evaluations to targets. Distinguished from area under possibly decreasing utility.

[17] Official abstract: test augmentation and previously undetected errors. Does not establish natural evaluator-correction histories.

[18] Official abstract: repository issue resolution through code edits. Future environment candidate only; no acquisition, execution or correction-history labels.

[19] Author abstract: interleaved reasoning and environment actions. Motivates recording tool/environment state; no direct recovery comparison.